

Adapting AlphaEvolve to Optimize Fully Homomorphic Encryption on TPUs

Shruthi Gorantala, Jianming Tong[†], Asra Ali, Baiyu Li, Jonathan Katz, Jeremy Kun
Thomas Steinke[‡], Abhradeep Thakurta[‡], Julian Walker[‡], Amir Yazdanbakhsh[‡]
Google [†]Georgia Institute of Technology [‡]Google DeepMind

Abstract—The deployment of Fully Homomorphic Encryption (FHE) at scale is hindered due to its heavy computational overhead. While specialized hardware accelerators like Google Tensor Processing Units (TPUs) can help, mapping complex cryptographic kernels onto such architectures remains a challenge. Efficient execution requires co-optimization between the systolic array-based Matrix Multiplication Unit (MXU) and Vector Processing Units (VPUs), as well as the orchestration of data movement across the vector register files. Existing compiler stacks often abstract low-level hardware utilization, requiring developers to adopt a manual trial-and-error process that often results in fragmented execution and underutilized resources.

To accelerate this development process, we use AlphaEvolve to automate the exploration of hardware-aware cryptographic-kernel optimizations. We frame optimization as an evolutionary search problem, utilizing the closed-loop system provided by AlphaEvolve, that leverages LLM-driven code generation. We use real-world feedback from hardware execution and rigorous correctness testing to guide the evolution process. We evaluate AlphaEvolve optimization on primitives for both the TFHE (Jaxite) and CKKS (CROSS) FHE schemes on Google Cloud TPUv5e, a contemporary TPU architecture. Within 24 hours of automated exploration, AlphaEvolve discovered implementation-level optimizations that improve TFHE bootstrap latency by $2.5\times$ and CKKS rotation and multiplication latency by $1.31\times$ and $1.18\times$, respectively, relative to human-engineered state of the art. These results demonstrate that AlphaEvolve can be used to enable researchers to navigate the optimization trade-offs between cryptography, compilers, and hardware accelerators.

I. INTRODUCTION

The rapid proliferation of personal data in AI ecosystems has created a demand for privacy-enhancing technologies (PETs). Fully Homomorphic Encryption (FHE) [9] stands as a foundational Privacy Enhancing Technology (PET) by enabling arbitrary computations on encrypted data without requiring decryption. Despite its privacy guarantees, Fully Homomorphic Encryption (FHE) incurs orders-of-magnitude compute and memory overheads, severely hindering its scalable deployment [10]. To mitigate these bottlenecks, the community has increasingly pivoted toward specialized accelerators like Tensor Processing Units (TPUs).

Mapping FHE primitives to accelerators requires engineering optimizations that spans across tiling workloads to strictly align with the underlying vector register granularity, navigating frontend abstractions (e.g., JAX and Pallas APIs for TPUs) to bypass costly, implicit layout transformations and sizing tensor granularities to effectively reduce off-chip memory latency. Modern compiler stacks like XLA [15] abstract these microarchitectural details. However performance tuning then

becomes a slow trial-and-error tuning loop where high-level code modification must traverse the compilation pipeline to observe its hardware impact. Current systems lack a systematic approach to map transformed cryptographic primitives onto commodity hardware designed for low-precision, GEMM-heavy workloads.

AI coding agents such as AlphaEvolve [14] have demonstrated significant advances in multiple fields including algorithm design, improving code efficiency, architecture discovery, and enabling discovery across scientific domains. Further, the use of AI to improve cryptographic implementation remains largely unexplored. This is primarily because LLM hallucinations might introduce security vulnerabilities which are unacceptable in cryptographic implementations.

In this paper, we use AlphaEvolve, an agentic framework, to optimize FHE kernels on hardware accelerators, such as Google’s TPU, with rigorous correctness checking. We formulate kernel optimization as an evolutionary search problem using AlphaEvolve, with real-world hardware feedback. By automating the discovery of micro-architectural improvements, AlphaEvolve enables cryptographic developers to explore architectural ideas and co-optimize cryptographic primitives on specialized hardware. The main contributions of our work are:

- We formulate JAX/Pallas-level FHE-on-TPU kernel optimizations as an evolutionary search problem.
- We design a hardware-in-the-loop evaluator that scores candidates using real TPU latency while rejecting candidates that fail robust functional correctness tests.
- We identify concrete optimization patterns discovered by the search that outperform domain-expert baselines: loop unrolling to reuse common parameters, finer-grained scheduling to hide off-chip memory access latency, minimize datatype conversion, and XLA-favorable tiling that improves VReg utilization.
- Within 24 hours of exploration, AlphaEvolve achieved a $2.5\times$ reduction in TFHE bootstrapping latency and speedups of $1.31\times$ and $1.18\times$ for CKKS rotation and multiplication, respectively.

II. BACKGROUND AND MOTIVATION

A. AlphaEvolve

AlphaEvolve [14] is an agentic framework designed for algorithmic and scientific discovery by combining LLMs with

the evolutionary search. It could be used to automatically evolve code in any programming language through a flywheel of code generation and code evaluation.

The workflow begins with a human programmer configuring the system based on expert-guided, task specific instructions, an initial code example to seed the evolutionary database and an evaluator that scores the generated code.

Next, the system samples code database and a pool of prompts. These prompts are fed into LLMs to generate new code modification. The generated code blocks are then compiled and fed into the evaluators, where they are scored, ranked, and only top-rank correct programs are added back to the evolutionary database. The ranking and evolution employs structured evolutionary techniques, such as island-based models or MAP-Elites algorithms [14]. These techniques balance genetic diversity based on inherent metrics (e.g., program length, code complexity) and performance on the evaluator metrics (fitness score). This optimization loop improves the code to find better program based on the fitness score.

B. Fully Homomorphic Encryption

FHE [9] enables computation on encrypted data. A homomorphic operation transforms ciphertexts so that the decrypted results are the same as if the operation were performed on the underlying plaintext data. Existing FHE schemes can be broadly categorized into scalar schemes and vector schemes based on the type of data they encrypt. Scalar schemes such as TFHE [5] encrypt bits or short integers, and supports general-purpose computation on encrypted data. Vector schemes such as CKKS [4] pack a vector of real or complex numbers into a ciphertext, and are widely used for privacy-preserving machine-learning and numerical workloads as vectors of data are processed in a SIMD fashion. Both approaches lead to 2~5 order-of-magnitude memory and computation overhead.

In this paper, we use AlphaEvolve to optimize implementations of both schemes on TPUs.

1) *TFHE - TFHE-bootstrap*: This is the key primitive in TFHE used in Google transpiler [11] and HEIR compiler [3]. TFHE bootstrap algorithms utilize the so-called Ring-GSW scheme defined over a polynomial residual ring $R_Q = \mathbb{Z}_Q[X]/(X^n + 1)$ and a special, cryptographic accumulator to perform decryption homomorphically. The primary bottleneck in the bootstrap operation is `blind_rotate`, which consists of a sequence of multiplication-accumulation operations on polynomials. Note that we reformulate polynomial multiplications in R_Q as Toeplitz matrix-vector products to take advantage of MXUs in TPUs.

2) *CKKS - CKKS-Rot and CKKS-Mult*: They perform homomorphic slot rotation and slot-wise multiplication, respectively, which are the backbone components in almost all CKKS use cases. We optimize their implementations from CROSS [16], combining automorphisms, key switching, rescaling, tensor multiplication, and modular reduction operations.

C. TPU Execution Model

Fig. 2 summarizes the TPU features. On-chip vector memory (VMEM) is partitioned across 128 lanes. Each lane contains

Algorithm: ToeplitzMul(**a**, **b**)

Input: $\mathbf{a} = (a_0, \dots, a_{n-1})$, $\mathbf{b} = (b_0, \dots, b_{n-1})$

Construct a Toeplitz matrix $T(\mathbf{a}) \in \mathbb{Z}^{n \times n}$:

$$T(\mathbf{a}) = \begin{bmatrix} a_0 & a_1 & \cdots & a_{n-1} \\ -a_{n-1} & a_0 & \cdots & a_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ -a_1 & -a_2 & \cdots & a_0 \end{bmatrix}$$

// Note: Negacyclic entries account for reduction modulo $X^n + 1$.

Compute the matrix-vector product $\mathbf{c} = T(\mathbf{a}) \cdot \mathbf{b} \in \mathbb{Z}_q^n$.

return $\mathbf{c}$ // Represents the polynomial $c(X) = \sum_{i=0}^{n-1} c_i X^i$.

Fig. 1: Polynomial multiplication $a(X) \cdot b(X) \in R_Q = \mathbb{Z}_Q[X]/(X^n + 1)$ via Toeplitz matrix-vector product, where $\mathbf{a}$ and $\mathbf{b}$ are coefficient vectors of $a(X)$ and $b(X)$, respectively.

8 sublanes, and each sublane provides 32 32-bit registers. Registers are organized into (8,128) blocks, called vector registers (VRegs), each formed by selecting one register from every sublane across all 128 lanes. The XLU performs data reorganization, while the VPU and MXU accelerate vector and matrix operations. Both computation and data movement execute in a single-instruction, multiple-data (SIMD) manner over one (8,128) VReg at a time. Consequently, programs that fully utilize VRegs achieve higher efficiency, whereas operations requiring fine-grained or irregular data access incur significant overhead.

For FHE, the implication is direct: mathematically equivalent implementations can differ sharply in performance depending on shapes, layout and scheduling choices of JAX/Pallas tensors.

D. The TPU Compiler Stack

We inherit the multi-layer TPU compilation stacks and profiling system to support FHE computations:

1) *JAX*: expresses high-level numerical and cryptographic computation. Both expert-crafted TPU libraries, Jaxite [2] and CROSS [16], use JAX / Pallas as the frontend.

2) *Pallas*: serves as a kernel language that allows for the creation of custom, low-level hardware kernels within JAX. Our system relies on Pallas kernels to establish fine-grained control over data movement and memory staging on the TPU [13].

3) *XLA*: lowers JAX/Pallas programs to TPU executables and applies optimizations such as fusion and layout selection [12], [15]. XLA compilation passes are shape-sensitive.

4) *Xprof*: is an integrated profiling tool within the XLA ecosystem that captures high-fidelity hardware execution traces. It extracts metrics such as compute and memory bandwidth utilization as well as on-chip and off-chip storage occupancy.

E. Challenge: Compiler-Obscured Performance Engineering

TPU performance engineering demands rigorous optimizations: (1) tiling workloads to strictly align with VReg granularities, (2) navigating frontend abstractions to bypass implicit layout transformations, and (3) sizing tensors to effectively hide off-chip memory latency behind compute.

Developers are strictly bound to high-level programming interfaces such as JAX and Pallas, limiting the easy access to manipulating the low level schedule. At this level of abstraction,

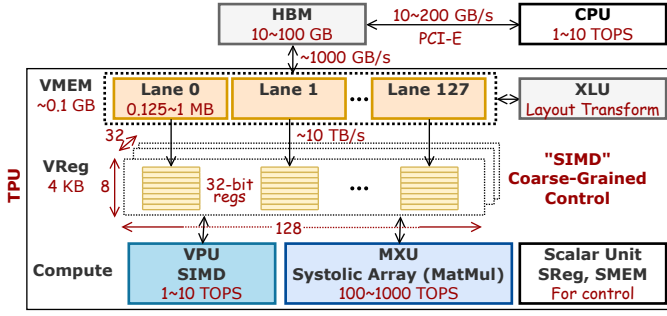


Fig. 2: TPU microarchitecture overview – SIMD machine.

every source modification must traverse hundreds of opaque compilation passes within stacks like XLA [15] before finally materializing as a low-level TPU execution trace. Because these intermediate layers obscure critical microarchitectural details, adapting an algorithm for TPU execution devolves into a sluggish trial-and-error process. Developers are forced to experimentally deduce the specific shapes, casts, loops, and tiling parameters at frontend JAX/Pallas that cause XLA to produce a good TPU program.

This paper adapts AlphaEvolve to automate this compiler-steering loop while preserving correctness and security guarantees.

The use of AI to improve cryptographic implementation is challenging as LLM hallucinations may introduce security vulnerabilities prone to side channel attacks. We use optimizing FHE as an initial case study where the data is encrypted.

III. ADAPTING ALPHAEVOLVE FOR CRYPTOGRAPHY

Configuring AlphaEvolve requires users to specify initial prompts (which establish a persona, algorithm and hardware platform), initial programs (which will be optimized by AlphaEvolve), the AlphaEvolve framework itself and an evaluation strategy (which validates the correctness, and then feeds reward as feedback back to AlphaEvolve).

Fig. 3 provides an overview of various parts of the system with details described below:

A. Initial Prompts

The initial prompts describe the task and role of AlphaEvolve as an expert in hardware-aware kernels engineering. The prompts also include a functional description of the cryptographic algorithms being optimized, architecture of the hardware accelerator, optimization strategies and a pool of potential optimization strategies.

B. Initial Program

We feed well-optimized reference codes into the evolutionary search framework as the initial programs. The choice of initial program needs a balance between providing enough code for AlphaEvolve to optimize while not allowing it to change the security guarantees provided by the algorithm. We use co-evolution as a process to evolve multiple kernels in tandem to find kernel improvements within various aspects of the cryptosystem. In addition to co-evolution, co-optimization can

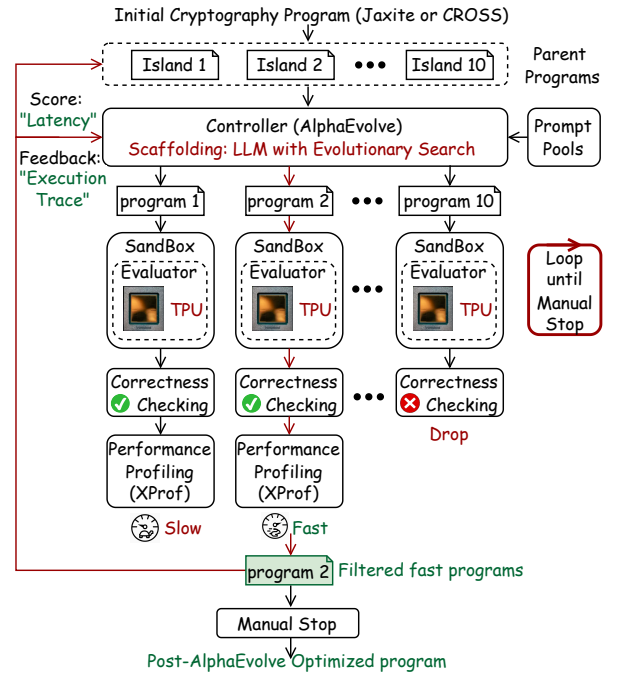


Fig. 3: Workflow for adapting AlphaEvolve to cryptographic kernel optimization. AlphaEvolve samples parent programs, prompts an LLM to generate candidate JAX/Pallas implementations, and evaluates each candidate in a sandbox on TPU hardware. Invalid candidates are rejected by correctness and security checks. Valid candidates are scored by latency and profiled through execution traces. Fast candidates are returned to the evolutionary population for further search.

be used to evolve security parameters and kernels, which allows for tuning security parameters for accelerator-specific kernel super-optimization.

C. Evaluation Engine

The evaluation engine only rewards candidate kernels that satisfy these multi-tiered correctness and security checks, thereby pruning invalid or insecure code early in the evolutionary search process.

1) *Correctness Checks*: To mitigate the risks of reward hacking and code hallucination inherent in LLM-generated kernels, we enforce strict correctness tests using randomized input vectors across all computational sub-layers. The correctness checking requires correct compilation to mitigate the risks of unintended bugs and erroneous code. This is done by testing subset of sub-modules as well as incorporating end-to-end tests which initializes the cryptosystem with valid security parameters, generates randomized inputs, encrypts the inputs, optionally runs cryptographic computations on the ciphertexts such as homomorphic computations for FHE cryptosystem, decrypts the resultant ciphertexts and checks against the expected plaintext outputs.

2) *Security Checks*: Our system validates any modifications to the parameters of the cryptosystem maintain standardized security guarantees (example: at least 128-bit security for

Fully Homomorphic Encryption utilizing acceptable parameters generated from industry standard lattice estimators [1]). Furthermore in case of fully homomorphic encryption, we implement safeguards to prevent the selection of scheme parameters that could lead to decryption failures by adding additional end-to-end test with high depth computation.

3) *Target Hardware Latency as Reward*: By deploying directly on the target platform, we use real-world latency as the primary reward score. In this paper, candidate kernels in Jaxite and CROSS are lowered through the XLA compiler and executed on physical TPU hardware. Our reward score uses microsecond-level execution latency of the resulting cryptographic primitive, excluding XLA compilation overhead to isolate runtime performance.

A persistent bottleneck in FHE acceleration is the lack of parity between isolated primitive speedups and actual end-to-end performance due to the bottlenecks introduced by data movement between host and device and also between devices. To ensure primitive kernel speedup translates to end-to-end speedup, AlphaEvolve evolves the one or more primitive kernels simultaneously, while using end-to-end latency across the entire functional module as the score. Measuring end-to-end latency proves to be a useful reward metric for co-evolution and co-optimization techniques.

4) *Feedback*: To facilitate the autonomous discovery of hardware-optimal execution patterns, our system captures high-fidelity execution traces during the evaluation cycle using xprof integrated with XLA. These metrics from profiling including compute and memory bandwidth utilization as well as on-chip and off-chip storage occupancy are fed back to AlphaEvolve as diagnostic feedback. By analyzing these traces, the agent can algorithmically identify latency bottlenecks and instances where the XLA compiler needs additional signal through JAX (or Pallas), enabling targeted iterative refinements to the cryptographic kernels.

D. Manual reviews / Functional equivalence

The best program generated by the evolutionary loop can be integrated into the original library after human review. Currently, a human programmer reviews the best program and merges the improved code into the original library. To reduce human-in-the-loop reviews, in principle a module to check functional equivalence of the initial code and the AlphaEvolve generated code could be used, though doing this is currently an unsolved research problem.

E. Putting it all together

After configuring initial prompts, initial programs, evaluation engine, and setting up appropriate hardware environment for the evaluators, AlphaEvolve begins the discovery process of automatically optimizing the cryptographic kernels. After reaching convergence based on the fitness function, the best generated program can be further tested for correctness through human reviews. The final code can then be merged into the original library and deployed for use.

IV. ALPHAEVOLVE FOR OPTIMIZING FHE

A. Setup

1) *Initial programs*: For TFHE, we use the bootstrap implementation from jaxite [2] and allow AlphaEvolve to modify various parts of `blind_rotate`. For CKKS, we adopt CKKS-Rot and CKKS-Mult from the TPU-based CKKS library, CROSS [16] that offers state-of-the-art throughput, as the initial programs and apply AlphaEvolve to search for faster implementations. We expose the implementations of RESCALE, TENSORMULTIPLY, KEYSWITCH, APPROXMODDOWN, and AUTOMORPHISM from both kernels to AlphaEvolve.

2) *Prompt Setup*: We use the system prompt together with a prompt sampled uniformly from §A. These prompts are designed to steer AlphaEvolve towards implementation-level improvements, especially scheduling and layout choices that increase VReg utilization in TPU and minimize implicit layout transformation, rather than toward new algorithms discovery.

3) *Correctness, and Performance Tests*: For TFHE, we incorporate end-to-end tests where the cryptosystem is initiated with parameters offering 128-bit security and random inputs are encrypted and evaluated against the generated code. The outputs are then decrypted and compared against the expected plaintext values. All values of the 3-bit lookup tables are evaluated for correctness.

For CKKS, functional correctness is enforced using a hidden test set with fixed ciphertext inputs and expected ciphertext outputs.

For both schemes, performance is measured from XProf traces, and we return the negative kernel latency as the optimization score so that lower latency corresponds to a higher score.

We run the evolution process with 10 controllers and 10 evaluators per controller, for a total of 100 TPUv5e chips.

TABLE I: Latency Comparison to SoTA [2], [16]

Workload	Latency Improvement	Code/Analysis
Blind Rotation (TFHE)	9.4 ms (TPUv5e-1)	Code: jaxite [2]
+ Unrolling	10 → 7.8 ms	Code: [6] Reason: §IV-B1
+ Fine-grained scheduling	7.8 → 6.3 ms	Code: [7] Reason: §IV-B2
+ Type Cast Removal	6.3 → 3.5 ms	Code: [8] Reason: §IV-B3
Post-AlphaEvolve blind Rotation	9.4 ms → 3.5 ms	2.85× speedup
Post-AlphaEvolve bootstrap	10 → 4 ms	2.5× speedup
CKKS-Mult (CKKS)	6340 μs (TPUv5e-1)	Code: ✓ [16]
+ XLA-favor tiling choice	5359 μs (TPUv5e-1)	Code: ✓ Reason: §IV-C
Post-AlphaEvolve CKKS-Mult	6340 → 5359 μs	1.18× speedup
CKKS-Rot (CKKS)	4854 μs (TPUv5e-1)	Code: ✓ [16]
+ XLA-favor tiling choice	3714 μs (TPUv5e-1)	Code: ✓ Reason: §IV-C
Post-AlphaEvolve CKKS-Rot	4854 → 3714 μs	1.3× speedup

B. AlphaEvolve for TFHE

AlphaEvolve explored several distinct classes of optimizations, ranging from standard loop transformations to complex kernel scheduling and data type optimizations.

1) *Loop Unrolling for Parameter Reuse* [6]: In the negacyclic vector-matrix polynomial multiplication (`negacyclic_vector_matrix_polymul`) routine [2], the system independently discovered that unrolling the primary for loop by a factor of 8 yielded significant latency reductions. In the baseline trace, the loop executes in serial, loading the common parameters repeatedly. By unrolling the loop, common parameters get reused across multiple loop

iterations. Because the target parameter size (3.51 MB) fit within the available on-chip memory, this transformation reduced latency from 10 ms to 7.8 ms. This highlights that AlphaEvolve could automatically identify and exploit loop unroll optimizations.

2) Fine-Grained Memory and Compute Scheduling [7]:

The system optimized data access patterns by adjusting the tiling and scheduling choice of the kernel. Specifically, it tiled one big JAX tensor into two blocks with shape halved. This modification allowed the XLA compiler to schedule more fine-grained data loading from off-chip memory, and hiding it behind computation. This optimization reduced the overall bootstrapping latency from 7.8 ms to 6.3 ms.

3) *Elimination of Redundant Type Casts [8]*: The most significant single optimization involved the removal of an explicit bitcast operation. The baseline implementation explicitly cast a right-hand side (`rhs`) operand to 8-bit integers (`i8`). AlphaEvolve identified that the dynamic range of the input data (values from 0 to 255 due to RLWE decomposition in [8]) was already sufficiently captured by the existing `bf16` precision. By removing the redundant and expensive bitcast operator, the system reduced the execution time from 6.29 ms to 3.35 ms. This optimization requires a deep understanding of both the mathematical dynamics of TFHE and the specific hardware execution costs of bitcast operations, making it difficult for human engineers to consistently identify.

C. AlphaEvolve for CKKS

- **CKKS-Rot**: On TPUv5e, AlphaEvolve reduces the latency of CKKS-Rot from 4874 μ s to 3754 μ s, yielding a $1.31\times$ speedup. This performance gain is primarily achieved by partitioning a tensor into two smaller sub-tensors. This strategic split triggers specialized XLA optimizations that significantly improve Vector Register (VReg) utilization during the AUTOMORPHISM stage. Consequently, AUTOMORPHISM’s contribution to the total end-to-end latency drops from 12% to just 4%. These results demonstrate AlphaEvolve’s ability to expose and exploit non-intuitive, compiler-specific performance “sweet spots” that are exceedingly difficult for human engineers to discover manually.
- **CKKS-Mult**: On TPUv5e, AlphaEvolve reduces latency of CKKS-Mult from 6340 μ s to 5359 μ s, corresponding to a $1.18\times$ speedup. Such performance improvement comes from the scheduling optimization to reuse common parameters. The modest improvement indicates that CKKS-Mult is less constrained by underlying compiler inefficiencies.

These results show that AlphaEvolve can improve CKKS kernel performance on TPU by discovering implementation changes that increase effective VReg utilization and reduce inefficiencies in the execution schedule.

D. Takeaways

The three workloads show that AlphaEvolve is most effective for cryptographic kernels when the bottleneck is caused by a mismatch between cryptographic data layout and the

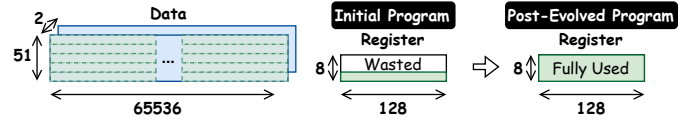


Fig. 4: Effect of the discovered layout changes. The initial program leaves substantial VReg capacity unused because operations are expressed at a granularity that does not match VReg. The evolved program exposes fuller VReg utilization.

TPU compiler/hardware execution grain. In TFHE, the search discovers both conventional locality improvements and a non-obvious cast elimination that depends on the ciphertext decomposition range. In CKKS, the search finds tensor shapes that steer XLA toward higher VReg utilization. Across all cases, the successful edits preserve the correctness and security while improving how the algorithm is presented to the compiler. By using AlphaEvolve to expose the performance headroom in the current TPU deployment flow, we also identify a key software bottleneck: existing programmable APIs force tensor reshapes and layout reorganizations that are not inherent to the hardware, introducing avoidable overhead.

V. CONCLUSION

This paper presents the first application of agentic AI to optimize FHE primitives on TPUs. We detail the design of using AlphaEvolve, a closed-loop system that combines LLMs with real-world hardware execution feedback and rigorous correctness checks. We demonstrate the framework’s ability to autonomously discover performance improvements on TPUs.

Our results demonstrate that AI-driven search uncovers optimizations frequently missed by domain experts.

A. Experience and Lessons Learned using AlphaEvolve

Some of our experiences and learnings in adapting AlphaEvolve for cryptographic optimization are described below:

1) *Granularity of Optimization*: Targeting isolated operations, such as polynomial multiplication (`polymul` [2]), failed to produce end-to-end speedups. We found that the agent requires visibility into broader operational scopes to make a meaningful impact on overall latency.

2) *Co-evolution and System-Level Scoring*: Building on the need for broader visibility, shifting to the co-evolution of the `external_product` and scoring it against the complete bootstrapping execution successfully reduced latency from 10 ms to 5.6 ms.

3) *Guided Domain Constraints*: While evolving the `blind-rotate` operation [2], the system discovered that RLWE decomposition allowed input tensors to safely use `int8` rather than `int32`, and that unrolling the loop by a factor of eight yielded significant benefits. This underscores the importance of domain experts selectively granting the AI permission to modify high-impact code blocks.

4) *Comprehensive Correctness Tests*: Initial attempts to evolve the AND-gate failed to cover all values of a 3-input lookup table (`lut3` [2]). We had to update the evaluator to exhaustively verify all lookup table values, demonstrating that

strict, comprehensive validation is essential to prevent the model from reward hacking.

5) *Security Checks*: Because performance cannot come at the expense of cryptographic integrity, we introduced a standardized parameter check using a lattice estimator to ensure at least 128-bit security when co-optimizing scheme parameters.

6) *Feedback and Prompts*: To effectively steer the evolutionary search, we integrated hardware profiling tools (`xprof`) directly into the feedback loop and provided hardware-specific optimization guidelines as automated compilation prompts.

B. Future Work

Our initial results focus on TPUs but the framework is extensible and can potentially optimize FHE primitives as well as any cryptographic primitives on a variety of hardware accelerators. It can also be integrated with simulators to guide specialized hardware-software co-design. To ensure security guarantees are maintained during hardware optimization, future iterations can integrate parameter selection with noise tracking. Furthermore, the bottleneck of manual code reviews can be reduced by adding functional equivalence checkers directly into the validation pipeline. Finally, a fundamental open question remains in the design of reward functions to safely allow AI agents to discover novel cryptographic algorithms and protocols while preserving security guarantees.

ACKNOWLEDGMENTS

This work represents an equal contributions between the first two authors, with contributing co-authors listed alphabetically. The authors express their gratitude to Charles Hong, Andrew Ferraiuolo, Ben Kreuter, and Cindee Madison for their valuable feedback during various phases of this work. We thank Po-Sen Huang, Ngan (NV) Vũ, and the AlphaEvolve team for their assistance in establishing the experimental framework. We thank Drew Angeloff, Ankita Malviya Bairaria, Naomi Black, Elie Burzstein, Bryant Gipson, Pankaj Rohatgi, Amanda Walker, Moti Yung and the Safeworks leadership team for their guidance throughout this project. Finally, we also thank our extended team at Google DeepMind for their support of this research direction.

We also acknowledge the use of large language models (LLMs) in the preparation of this manuscript. Specifically, the models were used for proofreading, summarizing text, and generating LaTeX outlines. The authors have reviewed and verified all generated content and take full responsibility for the final manuscript.

REFERENCES

- [1] M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *Cryptology ePrint Archive*, Paper 2015/046, 2015. [Online]. Available: <https://eprint.iacr.org/2015/046>
- [2] A. Ali, E. Astor, B. Gipson, S. Gorantala, M. Guevara, J. Kun, W. Lam, R. Misoczki, R. Springer, J. Takeshita, J. Tong, C. Tew, and C. Yun, “Jaxite: A fully homomorphic encryption backend targeting TPUs and GPUs, written in JAX,” <https://github.com/google/jaxite>, 2024.
- [3] A. Ali, J. Choi, B. Gipson, S. Gorantala, J. Kun, W. Legiest, L. Lim, A. Viand, M. Z. Demissie, and H. Zheng, “Heir: A universal compiler for homomorphic encryption,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.11095>

- [4] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science, vol. 10624. Springer, 2017, pp. 409–437.
- [5] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “TFHE: Fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, Jan 2020.
- [6] code perspective, “Add an unroll factor of 8 (reduces bootstrap speedup on tpu by 2 ms),” <https://github.com/google/jaxite/pull/88>, 2026, gitHub Pull Request #88, Accessed: 2026-05-06.
- [7] code perspective, “Improvements to the `vector_matrix_polymul` provides a speedup of 1.7 ms,” <https://github.com/google/jaxite/pull/89>, 2026, gitHub Pull Request #89, Accessed: 2026-05-06.
- [8] code perspective, “Update `_i32_matmul_unreduced_cggi` to use single `bfloat16` (instead of 4) for `int32` multiplications,” <https://github.com/google/jaxite/pull/90>, 2026, gitHub Pull Request #90, Accessed: 2026-05-06.
- [9] C. Gentry, “Fully homomorphic encryption using ideal lattices,” in *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, ser. STOC ’09. New York, NY, USA: Association for Computing Machinery, 2009, p. 169–178. [Online]. Available: <https://doi.org/10.1145/1536414.1536440>
- [10] S. Gorantala, R. Springer, and B. Gipson, “Unlocking the potential of fully homomorphic encryption,” *Communications of the ACM*, vol. 66, no. 6, pp. 68–75, jun 2023. [Online]. Available: <https://doi.org/10.1145/3582500>
- [11] S. Gorantala, R. Springer, S. Purser-Haskell, W. Lam, R. J. Wilson, A. Ali, E. P. Astor, I. Zukerman, S. Ruth, C. Dibak, P. Schoppmann, S. Kulankhina, A. Forget, D. Marn, C. Tew, R. Misoczki, B. Guillen, X. Ye, D. Kraft, D. Desfontaines, A. Krishnamurthy, M. Guevara, I. M. Perera, Y. Sushko, and B. Gipson, “A general purpose transpiler for fully homomorphic encryption,” *CoRR*, vol. abs/2106.07893, 2021. [Online]. Available: <https://arxiv.org/abs/2106.07893>
- [12] X. He, “Accelerated linear algebra compiler for computationally efficient numerical models: Success and potential area of improvement,” *PLOS ONE*, vol. 18, no. 2, p. e0282265, 2023.
- [13] JAX Developers, “Pallas quickstart — JAX documentation,” <https://docs.jax.dev/en/latest/pallas/quickstart.html>, 2024.
- [14] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian *et al.*, “Alphaevolve: A coding agent for scientific and algorithmic discovery,” *arXiv preprint arXiv:2506.13131*, 2025.
- [15] OpenXLA Project, “OpenXLA: Accelerated linear algebra,” <https://github.com/openxla/xla>, 2026.
- [16] J. Tong, T. Huang, J. Dang, L. de Castro, A. Itagi, A. Golder, A. Ali, J. Jiang, J. Kun, Arvind, G. E. Suh, and T. Krishna, “Leveraging asic ai chips for homomorphic encryption,” ser. HPCA’26. Australia: 2026 IEEE International Symposium on High Performance Computer Architecture (HPCA), 2026.

APPENDIX

A. LLM Instruction-Prompt Pool

- **Prompt 1**: Suggest a new way to minimize data reorganization in the program.
- **Prompt 2**: Suggest a new way to keep the logical shapes of JAX arrays invariant throughout execution.
- **Prompt 3**: Suggest a new way to hide off-chip memory latency behind computation.
- **Prompt 4**: Suggest a new optimization idea based on your expert knowledge of TPU performance tuning.

B. Role and Hardware Architecture Prompts

- **Role**: You are an expert hardware-aware compiler optimizer specializing in TPU architectures.
- **Objective**: Optimize the input computational graph for maximum performance on the TPU v5e architecture. You must make strategic decisions regarding operation mapping (Vector vs. Matrix units), memory hierarchy management, and parallelism.

- **Target Hardware Specifications:**
 - **Compute Capabilities (Peak):**
 - Clock Frequency: 1.50 GHz
 - Matrix Units (MXU): 197 TFLOPS (bf16/fp8), 394 TOPS (int8), 788 TOPS (int4).
 - Vector Units (VPU): vmatmul (1.54 TB/s), vmatpush (6.16 TB/s).
 - **Memory Constraints (CRITICAL):**
 - CMEM: None (0 bytes). Do not tile data into CMEM.
 - Vmem (Vector Memory): 128 MB. Read: 18.5 TB/s, Write: 6.16 TB/s.
 - HBM (High Bandwidth Memory): ~17.2 GB at 820 GB/s.
 - **Interconnect:** 2D Torus, 4 links/chip, 45 GB/s per link.
- **Optimization Strategies Required:**
 - **Compute Mapping:** Map matrix multiplications to the Systolic Array (MXU). Map element-wise, reduction, and non-matmul ops to the Vector Unit (VPU).
 - **Memory Tiling & Fusion:** All functional tiling must target the 128 MB Vmem. Perform loop fusion to avoid spilling back to HBM.
 - **Precision Tuning:** Evaluate suitability for int8 quantization for 2× throughput.
 - **Pipeline Parallelism:** Overlap HBM-to-Vmem DMA transfers with VPU/MXU execution.
- **Output Requirements:** Generate optimized code/IR and summarize: theoretical peak utilization, memory bandwidth utilization, and a bottleneck analysis.